

Estruturas de Aceleração

INF2604 – Fundamentos de Computação Gráfica

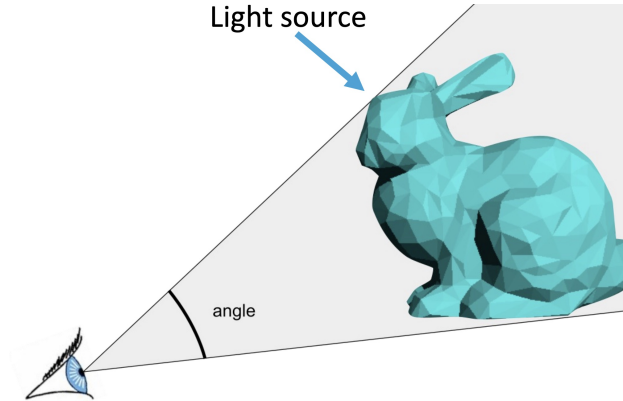
Waldemar Celes
celes@inf.puc-rio.br

Departamento de Informática, PUC-Rio



Instanciação de malhas de triângulos

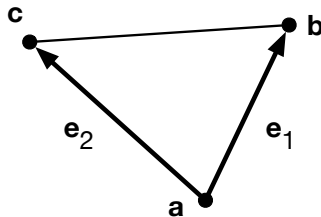
Como calcular a interseção do raio com a malha?



Instanciação de triângulos

Representação de triângulo

- ▶ Vértices no sentido anti-horário: $\mathbf{a}, \mathbf{b}, \mathbf{c}$
- ▶ Arestas do triângulo: $\vec{\mathbf{e}}_1 = \mathbf{b} - \mathbf{a}$, $\vec{\mathbf{e}}_2 = \mathbf{c} - \mathbf{a}$
- ▶ Área do triângulo: $A = \frac{|\vec{\mathbf{e}}_1 \times \vec{\mathbf{e}}_2|}{2}$



Instanciação de triângulos

Coordenadas baricêntricas

- ▶ Ponto no interior do triângulo
- ▶ Coordenadas baricêntricas

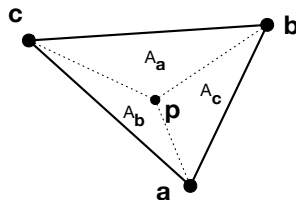
$$\mathbf{p} = w \mathbf{a} + u \mathbf{b} + v \mathbf{c}$$

onde:

$$w = \frac{A_{\mathbf{a}}}{A}, \quad u = \frac{A_{\mathbf{b}}}{A}, \quad v = \frac{A_{\mathbf{c}}}{A}$$

$$w + u + v = 1$$

$$\therefore \mathbf{p} = (1 - u - v) \mathbf{a} + u \mathbf{b} + v \mathbf{c}$$



Instanciação de triângulos

Interseção raio-triângulo

- ▶ Algoritmo de Möller-Trumbore (1997)
 - ▶ Ponto no triângulo

$$\mathbf{p} = (1 - u - v) \mathbf{a} + u \mathbf{b} + v \mathbf{c}$$

- ▶ Expandindo:

$$\mathbf{p} = \mathbf{a} - u \mathbf{a} - v \mathbf{a} + u \mathbf{b} + v \mathbf{c}$$

$$\mathbf{p} = \mathbf{a} + u(\mathbf{b} - \mathbf{a}) + v(\mathbf{c} - \mathbf{a})$$

$$\mathbf{p} = \mathbf{a} + u\vec{\mathbf{e}}_1 + v\vec{\mathbf{e}}_2$$

- ▶ Substituindo $\mathbf{p}$ pela expressão do raio

$$\mathbf{o} + t\hat{\mathbf{d}} = \mathbf{a} + u\vec{\mathbf{e}}_1 + v\vec{\mathbf{e}}_2$$

$$\mathbf{o} - \mathbf{a} = -t\hat{\mathbf{d}} + u\vec{\mathbf{e}}_1 + v\vec{\mathbf{e}}_2$$



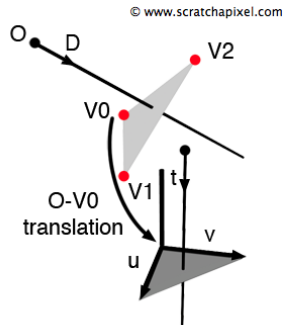
Interseção raio-triângulo

Algoritmo de Möller-Trumbore (1997)

- Em forma matricial

$$\begin{bmatrix} -\hat{\mathbf{d}} & \vec{\mathbf{e}}_1 & \vec{\mathbf{e}}_2 \end{bmatrix} \begin{bmatrix} t \\ u \\ v \end{bmatrix} = \mathbf{o} - \mathbf{a}$$

- Recai na resolução de um sistema linear 3×3



Interseção raio-triângulo

Regrama de Cramer

$$A \mathbf{x} = \mathbf{b}$$

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix}$$

► Tem-se:

$$x_0 = \frac{|A_0|}{|A|}, \quad x_1 = \frac{|A_1|}{|A|}, \quad x_2 = \frac{|A_2|}{|A|}$$

onde:

$$A_0 = \begin{bmatrix} b_0 & a_{01} & a_{02} \\ b_1 & a_{11} & a_{12} \\ b_2 & a_{21} & a_{22} \end{bmatrix}$$

$$A_1 = \dots, \quad A_2 = \dots$$



Interseção raio-triângulo

Determinante e produto triplo

$$A = \begin{bmatrix} \vec{a} & \vec{b} & \vec{c} \end{bmatrix}$$
$$|A| = \begin{vmatrix} \vec{a} & \vec{b} & \vec{c} \end{vmatrix} = \vec{a} \cdot (\vec{b} \times \vec{c})$$



Interseção raio-triângulo

Determinante e produto triplo

$$A = \begin{bmatrix} \vec{a} & \vec{b} & \vec{c} \end{bmatrix}$$
$$|A| = \begin{vmatrix} \vec{a} & \vec{b} & \vec{c} \end{vmatrix} = \vec{a} \cdot (\vec{b} \times \vec{c})$$

► Propriedades

- $\vec{a} \cdot (\vec{b} \times \vec{c}) = \vec{b} \cdot (\vec{c} \times \vec{a}) = \vec{c} \cdot (\vec{a} \times \vec{b})$
- $\vec{a} \cdot (\vec{b} \times \vec{c}) = (\vec{a} \times \vec{b}) \cdot \vec{c}$
- $\vec{a} \cdot (\vec{b} \times \vec{c}) = -\vec{a} \cdot (\vec{c} \times \vec{b})$



Interseção raio-triângulo

Voltando ao nosso sistema

$$\begin{bmatrix} -\hat{\mathbf{d}} & \vec{\mathbf{e}}_1 & \vec{\mathbf{e}}_2 \end{bmatrix} \begin{bmatrix} t \\ u \\ v \end{bmatrix} = \vec{\mathbf{r}}, \quad \text{com: } \vec{\mathbf{r}} = \mathbf{o} - \mathbf{a}$$

► Aplicando regra de Cramer

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\begin{vmatrix} -\hat{\mathbf{d}} & \vec{\mathbf{e}}_1 & \vec{\mathbf{e}}_2 \end{vmatrix}} \begin{bmatrix} \begin{vmatrix} \vec{\mathbf{r}} & \vec{\mathbf{e}}_1 & \vec{\mathbf{e}}_2 \end{vmatrix} \\ \begin{vmatrix} -\hat{\mathbf{d}} & \vec{\mathbf{r}} & \vec{\mathbf{e}}_2 \end{vmatrix} \\ \begin{vmatrix} -\hat{\mathbf{d}} & \vec{\mathbf{e}}_1 & \vec{\mathbf{r}} \end{vmatrix} \end{bmatrix}$$

► Usando produto triplo

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1)} \begin{bmatrix} \vec{\mathbf{r}} \cdot (\vec{\mathbf{e}}_1 \times \vec{\mathbf{e}}_2) \\ \hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{r}}) \\ \hat{\mathbf{d}} \cdot (\vec{\mathbf{r}} \times \vec{\mathbf{e}}_1) \end{bmatrix}$$



Interseção raio-triângulo

Algoritmo de Möller-Trumbore (1997)

- ▶ Observações com relação ao valor do denominador
 - ▶ Se $\hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1) = 0$, não há interseção
 - ▶ De fato, raio paralelo a triângulo: $\hat{\mathbf{d}} \perp \vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1$
 - ▶ Se $\hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1) > 0$, é *front facing*
 - ▶ De fato, já que $-\vec{\mathbf{n}} = \vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1$
 - ▶ Se $\hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1) < 0$, é *back facing*

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1)} \begin{bmatrix} \vec{\mathbf{r}} \cdot (\vec{\mathbf{e}}_1 \times \vec{\mathbf{e}}_2) \\ \hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{r}}) \\ \hat{\mathbf{d}} \cdot (\vec{\mathbf{r}} \times \vec{\mathbf{e}}_1) \end{bmatrix}$$



Interseção raio-triângulo

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1)} \begin{bmatrix} \vec{\mathbf{r}} \cdot (\vec{\mathbf{e}}_1 \times \vec{\mathbf{e}}_2) \\ \hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{r}}) \\ \hat{\mathbf{d}} \cdot (\vec{\mathbf{r}} \times \vec{\mathbf{e}}_1) \end{bmatrix}$$

Algoritmo de Möller-Trumbore (1997)

- ▶ Observações com relação ao valor do denominador
 - ▶ Se $\hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1) = 0$, não há interseção
 - ▶ De fato, raio paralelo a triângulo: $\hat{\mathbf{d}} \perp \vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1$
 - ▶ Se $\hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1) > 0$, é *front facing*
 - ▶ De fato, já que $-\vec{\mathbf{n}} = \vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1$
 - ▶ Se $\hat{\mathbf{d}} \cdot (\vec{\mathbf{e}}_2 \times \vec{\mathbf{e}}_1) < 0$, é *back facing*
- ▶ Implementação
 1. Calcule o denominador: verifique as observações acima
 2. Calcule u : se $u < 0$ ou $u > 1$, não há interseção
 3. Calcule v : se $v < 0$ ou $v > 1$ ou $u + v > 1$, não há interseção
 4. Calcule t : se $t < 0$, não há interseção



Malhas de triângulos

Representação

- Tabela de vértices

$$v : \quad \mathbf{p} \quad \hat{\mathbf{n}}$$

- Tabela de incidência

$$t : \quad v_0 \quad v_1 \quad v_2$$

Malhas de triângulos

Representação

- ▶ Tabela de vértices

$$v : \quad \mathbf{p} \quad \hat{\mathbf{n}}$$

- ▶ Tabela de incidência

$$t : \quad v_0 \quad v_1 \quad v_2$$

Cálculo de interseção raio-malha

- ▶ Necessidade de estrutura de aceleração

Estrutura espacial de aceleração

Grade regular

- ▶ Subdivide o domínio (da cena ou do objeto) em células regulares

Construção

- ▶ Escolha uma resolução para a grade (crítico)
- ▶ Para cada célula, armazene uma lista dos triângulos que a interceptam

Traçado de raio

- ▶ Traçado eficiente nas células da grade
- ▶ Determina interseção do raio com triângulos de cada célula
 - ▶ Naturalmente acha a interseção mais próxima

Grade regular

Construção

- ▶ Para cada triângulo:
 - ▶ Determine sua caixa envolvente
 - ▶ Marque as células que interceptam a caixa envolvente
- ▶ Ajuste fino
 - ▶ Para cada célula marcada, verifique se existe **eixo separador**
 - ▶ Projeção da célula e do triângulo no eixo não se sobrepõem
 - ▶ Candidatos a eixo separador (13 ao total):
 - ▶ A normal do triângulo
 - ▶ Os 3 eixos da célula
 - ▶ Os 9 produtos vetoriais entre arestas do triângulo e eixos da célula

Grade regular

Traçado de raio

► Raio

$\mathbf{r}(t) = \mathbf{o} + t \mathbf{d}$, com origem $\mathbf{o} = (o_x, o_y, o_z)$ e direção $\mathbf{d} = (d_x, d_y, d_z)$

► Dimensão de células da grade

h_x, h_y, h_z

Grade regular

Traçado de raio

- ▶ Determina célula inicial
 - ▶ Se origem dentro da grade

$$i = \left\lfloor \frac{o_x - x_{\min}}{h_x} \right\rfloor, \quad j = \left\lfloor \frac{o_y - y_{\min}}{h_y} \right\rfloor, \quad k = \left\lfloor \frac{o_z - z_{\min}}{h_z} \right\rfloor$$

- ▶ Se origem fora da caixa
 - ▶ Calcula interseção de raio com caixa

$$\mathbf{o}' = \mathbf{p}_{inter} + \epsilon \mathbf{d}$$

Grade regular

Traçado de raio

- ▶ Para cada eixo da grade, determina se raio está na direção positiva ou negativa
 - ▶ $\Delta_i = d_x > 0 ? +1 : -1$
 - ▶ $\Delta_j = d_y > 0 ? +1 : -1$
 - ▶ $\Delta_k = d_z > 0 ? +1 : -1$
- ▶ Incremento de coordenada até próxima célula

$$\delta_x = \frac{h_x}{|d_x|} \quad \delta_y = \frac{h_y}{|d_y|} \quad \delta_z = \frac{h_z}{|d_z|}$$

- ▶ Próxima interseção

$$t_x = \frac{x_{next} - o_x}{d_x}, \text{ onde } x_{next} \text{ é a face à direita } (d_x > 0) \text{ ou esquerda } (d_x < 0)$$

- ▶ Análogo para y e z



Grade regular

Traçado de raio

- ▶ Algoritmo
 - ▶ Processa célula atual (i, j, k)
 - ▶ Calcula interseção
 - ▶ Avança para próxima célula
 - ▶ Repete enquanto dentro da grade

- ▶ **obs:** se $d_x = 0$, fazer:

$$t_x = +\text{inf}$$

$$\delta_x = +\text{inf}$$

```
if  $t_x < t_y$  then
  if  $t_x < t_z$  then
     $i = i + \Delta_i$ 
     $t_x = t_x + \delta_x$ 
  else
     $k = k + \Delta_k$ 
     $t_z = t_z + \delta_z$ 
else
  if  $t_y < t_z$  then
     $j = j + \Delta_j$ 
     $t_y = t_y + \delta_y$ 
  else
     $k = k + \Delta_k$ 
     $t_z = t_z + \delta_z$ 
```



Estrutura espacial de aceleração

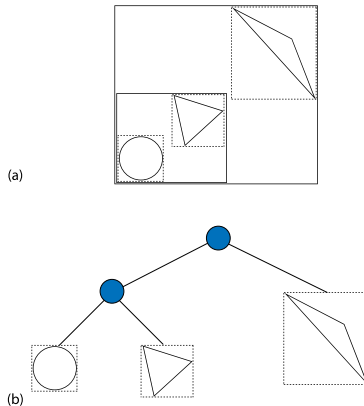
Grade regular

- ▶ Estrutura simples, percorrimento simples
- ▶ Sensível à distribuição dos dados
 - ▶ Ineficiente para cenas esparsas
 - ▶ Ineficiente para objetos com grandes triângulos
- ▶ Melhoria: uso de tabela de dispersão (*hash*)
 - ▶ Reduz espaço de armazenamento de grades esparsas
 - ▶ Não resolve ineficiência
 - ▶ Todas as células ao longo do raio têm que ser processadas

Estrutura espacial de aceleração

Estrutura hierárquica

► Árvore de caixas envolventes



Estrutura espacial de aceleração

Construção

1. Calcula a caixa envolvente de cada primitiva
2. Constrói árvore (binária) agrupando primitivas
3. Transforma árvore numa representação compacta (recomendado)

Estrutura espacial de aceleração

Construção

1. Calcula a caixa envolvente de cada primitiva
2. Constrói árvore (binária) agrupando primitivas
3. Transforma árvore numa representação compacta (recomendado)

Estrutura hierárquica (árvore)

- ▶ Nós internos: dois (ou mais) nós filhos (não têm primitivas)
- ▶ Nós externos (folhas): uma ou mais primitivas

Estrutura espacial de aceleração

Construção

1. Calcula a caixa envolvente de cada primitiva
2. Constrói árvore (binária) agrupando primitivas
3. Transforma árvore numa representação compacta (recomendado)

Estrutura hierárquica (árvore)

- ▶ Nós internos: dois (ou mais) nós filhos (não têm primitivas)
- ▶ Nós externos (folhas): uma ou mais primitivas

Cálculo de interseção

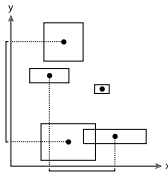
- ▶ Para cada nó
 - ▶ Se tem interseção raio-caixa
 - ▶ Processa os dois nós filhos, se nó interno
 - ▶ Calcula interseção com primitivas, se nó folha



Estrutura espacial de aceleração

Algoritmo de construção (árvore binária)

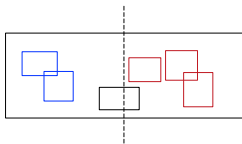
- ▶ Calcula centróide das caixas envolventes das primitivas
- ▶ Calcula caixa envolvente dos centróides
- ▶ Subdivide eixo de maior comprimento
 - ▶ Distribui primitivas entre as duas subdivisões
- ▶ Repete o mesmo procedimento para cada uma das subdivisões



Estrutura espacial de aceleração

Critério de agrupamento de primitivas

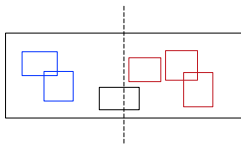
- ▶ Subdivisão do espaço
 - ▶ Primitivas podem estar presentes em mais de um nó



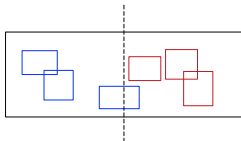
Estrutura espacial de aceleração

Critério de agrupamento de primitivas

- ▶ Subdivisão do espaço
 - ▶ Primitivas podem estar presentes em mais de um nó

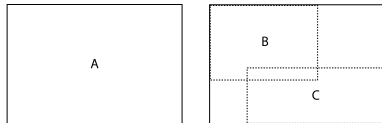


- ▶ Subdivisão de primitivas
 - ▶ Primitivas presentes em apenas um nó



Subdivisão de primitivas

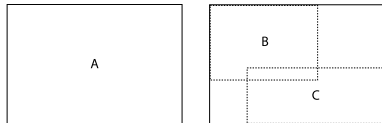
Critério para determinação do plano de corte



Subdivisão de primitivas

Critério para determinação do plano de corte

- ▶ Distribuição uniforme

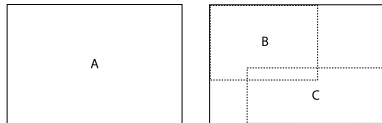


- ▶ Heurística da área da superfície (*SAH - Surface area heuristic*)

Subdivisão de primitivas

Critério para determinação do plano de corte

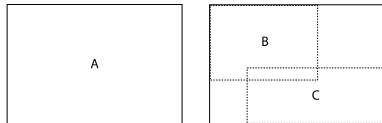
- ▶ Distribuição uniforme
 - ▶ Os dois nós filhos com o mesmo número de primitivas
 - ▶ Acha a primitiva mediana na ordenação e subdivide
 - ▶ Em C++, use `std::nth_element`
- ▶ Heurística da área da superfície (*SAH - Surface area heuristic*)



Subdivisão de primitivas

Critério para determinação do plano de corte

- ▶ Distribuição uniforme
 - ▶ Os dois nós filhos com o mesmo número de primitivas
 - ▶ Acha a primitiva mediana na ordenação e subdivide
 - ▶ Em C++, use `std::nth_element`
- ▶ Heurística da área da superfície (*SAH - Surface area heuristic*)
 - ▶ Busca minimizar custo de calcular a interseção do nó com dois filhos



$$cost(B, C) = t_{trav} + p_B \sum_{i=1}^{N_B} t_{isect}(b_i) + p_C \sum_{i=1}^{N_C} t_{isect}(c_i)$$

- ▶ t_{trav} : tempo para processamento do nó
- ▶ t_{isect} : tempo para cálculo de interseção de cada primitiva
- ▶ p_B, p_C : probabilidades do raio passar em cada filho



Heurística da área da superfície

De probabilidade geométrica

- ▶ A probabilidade de um raio (aleatório, uniformemente distribuído) que intercepta a caixa A também interceptar a caixa $B \subset A$ é dada por:

$$p(B|A) = \frac{s_B}{s_A}$$

- ▶ onde s_A e s_B são as áreas das caixas

Heurística da área da superfície

De probabilidade geométrica

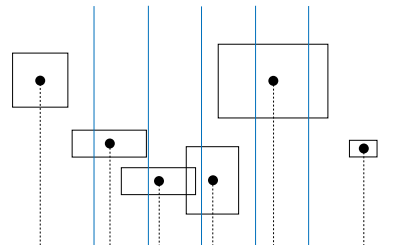
- ▶ A probabilidade de um raio (aleatório, uniformemente distribuído) que intercepta a caixa A também interceptar a caixa $B \subset A$ é dada por:

$$p(B|A) = \frac{s_B}{s_A}$$

- ▶ onde s_A e s_B são as áreas das caixas

Usamos essa probabilidade para estimar o custo

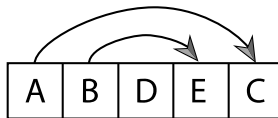
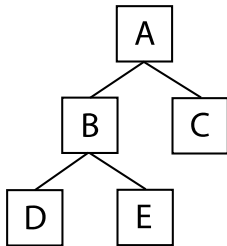
- ▶ Assumindo $t_{isect}(b_i) = t_{isect}(c_i) = k$
 - ▶ Recai em minimizar $p_B N_B + p_C N_C$
- ▶ Podemos avaliar alguns poucos pontos de corte, ao invés dos $n - 1$ possíveis



Compactação da árvore de nós

Arranjo linear da estrutura hierárquica

- ▶ Nós armazenados linearmente na ordem *depth-first*
- ▶ Primeiro filho segue nó pai
- ▶ Segundo filho via *offset*
- ▶ Folhas não têm *offset*



Cálculo de interseção

Algoritmo para percorrer hierarquia

```
BVH.Intersect(ray)  
  S = EmptyStack   nodes to be processed  
  S.push(root)  
  while not S.IsEmpty() do  
    node = S.Pop()   node being processed  
    if node.Intersect(ray) then  
      if node.IsLeaf() then  
        t = node.IntersectPrimitive()  
        tmin = min(tmin, t)  
      else  
        S.Push(node.left)  
        S.Push(node.right)  
  return tmin
```

